

SAHAYAK - MASTER PROJECT CONTEXT

Project Overview

SahayaK is a hyperlocal volunteer-assistance platform that connects elderly citizens, disabled individuals, and digitally excluded citizens with nearby volunteers who can help them complete government, banking, healthcare, education, and documentation-related processes.

This is NOT:

- A donation platform
- A charity platform
- A freelance marketplace
- A gig economy platform

This IS:

- A civic assistance platform
- A community volunteer platform
- A social impact platform

Core Mission:

"Donate time and knowledge instead of money."

Volunteers help citizens complete important processes and earn recognition through a Help Score system.

Target Users

Seeker

People who need help:

- Senior citizens
- Disabled individuals
- Citizens with low digital literacy
- Citizens struggling with government services

Examples:

- Applying for pension
- Updating Aadhaar
- Applying for disability schemes
- Opening Jan Dhan accounts

- Applying for government benefits
-

Volunteer

People willing to help:

- Students
- Young professionals
- Community volunteers

Motivation:

- Social impact
 - Community service
 - Help Score recognition
-

Admin

Platform management:

- Manage users
 - Manage schemes
 - Monitor activity
 - Moderate content
 - Manage reports
-

Technology Stack

Framework:

- Next.js 15
- App Router

Language:

- TypeScript

Styling:

- Tailwind CSS

Backend:

- Supabase

Maps:

- Leaflet
- OpenStreetMap

Authentication:

- Supabase Auth

Database:

- PostgreSQL via Supabase

Storage:

- Supabase Storage

Realtime:

- Supabase Realtime

Architecture Rules

IMPORTANT

Do NOT create:

- Express server
- NestJS server
- Custom backend server

Frontend communicates directly with Supabase.

Architecture:

Next.js ↓ Supabase ↓ Database

Existing Supabase Database

Database already exists.

Do NOT generate migrations.

Do NOT redesign schema.

Use existing database.

Tables already available:

- profiles
- tasks
- task_messages
- notifications
- help_score_log
- schemes

Storage bucket:

- task-proofs

Realtime enabled:

- tasks
- notifications
- task_messages

User Roles

seeker

Can:

- Create tasks
- View own tasks
- Chat with volunteer
- Verify completion

Cannot:

- Accept tasks

volunteer

Can:

- Browse open tasks
- Accept tasks
- Upload proof
- Earn Help Score

Cannot:

- Create help requests
-

admin

Can:

- Access admin dashboard
 - Manage users
 - Manage schemes
 - View analytics
-

Task Lifecycle

Status values:

open accepted in_progress completed verified

Flow:

Seeker creates task ↓ Volunteer accepts task ↓ Volunteer works on task ↓ Volunteer marks completed ↓
Seeker verifies ↓ Help Score awarded

Main Features

Authentication

- Register
 - Login
 - Logout
 - Session persistence
 - Role-based access
-

Profile Management

- Name
- Phone
- Location
- Role
- Profile details

Help Requests

Create help request:

- Title
- Description
- Category
- Location
- Urgency

View:

- Open requests
 - Accepted requests
 - Completed requests
-

Volunteer Feed

Browse nearby requests.

Filter by:

- Category
- Location
- Urgency

Accept task.

Task Details

Display:

- Request details
 - Volunteer details
 - Chat
 - Status timeline
 - Completion proof
-

Chat

Realtime messaging.

Participants:

- Seeker
- Volunteer

Use task_messages table.

Notifications

Realtime notifications.

Examples:

- Task accepted
- New message
- Task completed
- Task verified

Use notifications table.

Help Score

Volunteer receives points after verification.

Display:

- Total score
- History
- Contributions

Use help_score_log table.

Government Schemes Directory

Use schemes table.

Display:

- Scheme name
- Description
- Eligibility
- Category

Searchable.

Filterable.

Design Requirements

Accessibility first.

Primary users include elderly citizens.

Requirements:

- Large typography
- Large buttons
- High contrast
- Simple forms
- Clear navigation
- Mobile first
- Easy readability

Avoid:

- Complex dashboards
 - Small text
 - Dense layouts
-

Internationalization

Architecture should support future Gujarati translation.

Do not hardcode language assumptions.

Keep text structure translation-ready.

Next.js Folder Structure

Use:

```
src/ | ├── app/ | ├── components/ | ├── services/ | ├── hooks/ | ├── context/ | ├──  
lib/ | ├── types/ | ├── constants/ | ├── utils/ | └── middleware.ts
```

Service Layer Rules

All Supabase queries must be inside services.

Examples:

services/ ├── auth.service.ts ├── task.service.ts ├── profile.service.ts ├── scheme.service.ts
├── notification.service.ts └── storage.service.ts

Pages and components should never directly query Supabase.

Route Structure

Public:

/ /about /how-it-works /schemes /login /register

Seeker:

/seeker/dashboard /seeker/tasks /seeker/profile

Volunteer:

/volunteer/dashboard /volunteer/tasks /volunteer/profile

Admin:

/admin/dashboard /admin/users /admin/schemes /admin/reports

Route Protection

Use middleware.

Requirements:

- Unauthenticated users redirected to login
 - Role-based access
 - Admin routes protected
 - Volunteer routes protected
 - Seeker routes protected
-

Deliverables

Generate:

1. Complete production-grade folder structure
2. Supabase integration
3. TypeScript types
4. Auth system
5. Role-based routing
6. Layout system
7. Reusable UI component library
8. Dashboard architecture
9. Service layer architecture
10. Scalable codebase

Prioritize architecture before page implementation.

Build production-ready foundations first.